Capability Envelopes: Deployable Budget Enforcement for Unmodified Linux Binaries

Anonymous Author(s)

Abstract

Capability systems promise least-privilege execution but have seen minimal adoption in production Linux environments because they demand source-code modification—a "rewrite mandate" that is prohibitive for legacy binaries. We observe that while binary permissions are hard to deploy retroactively, *quantified budgets* are not: a budget can be attached to any process at launch time without modifying its code. We present *capability envelopes*, a kernel-level mechanism that wraps unmodified Linux binaries in programmable budget policies governing type membership, direction enforcement, data volume, operation count, and temporal expiry.

We implement capability envelopes in A20OS, a research operating system with a dual-ABI architecture that runs unmodified musl-linked binaries alongside native capability-aware services. Our kernel mediator enforces budget accounting across ten escape-surface classes covering 366 registered system calls, with per-direction right checks on tracked shadow handles. A 20-cell pilot experiment across four defense configurations (unrestricted, Landlock-permissive, Landlock-strict, envelope) demonstrates the coarse-grained dilemma: permissive path-based policies admit all tested attacks while strict variants block legitimate installs. Within this pilot, only the envelope arm simultaneously completes the benign installation and blocks all four tested attack classes, through type denial, rights clamping, and budget exhaustion. An exact-execution experiment runs the unmodified install scripts of 66 real malicious packages on stock Node.js and CPython under A20OS: every one of the 25 samples that attempted a network operation is blocked at `socket()` under an install-time envelope, while the same payloads reach live C2 infrastructure undefended; a complementary canonical-replay study covers 87 samples with identical conclusions. Overhead microbenchmarks under full-system emulation show $+4.1\%$ on the acquisition path and $+17.7$–29.2% on the file use path.

1 Introduction

Capability-based security systems—from Capsicum [1] to Landlock [8]—promise least-privilege execution by replacing ambient authority with explicit capabilities. Despite four decades of research, adoption in production Linux remains negligible. The root cause is what we call the *rewrite mandate*: enjoying capability discipline requires modifying source code to replace global namespace accesses with capability-constrained operations. For legacy binaries, proprietary software, and third-party packages, this mandate is prohibitive.

We observe an asymmetry that suggests a path forward. Binary permissions ("this process may open any file") are difficult to constrain retroactively because they are woven into the program's logic. Quantified budgets ("this process may perform at most N file operations within T seconds"), by contrast, can be attached at deployment time as a wrapper policy—the binary itself needs no modification. We call this property *budget deployability*.

1.1 Capability Envelopes

We present *capability envelopes*: a kernel-level mechanism that wraps unmodified Linux binaries in programmable budget policies governing five dimensions:

1. **Type membership**—which resource classes (file, socket, pipe, memory) the process may access;
2. **Rights ceiling**—per-class maximum permission bits (read, write, stat, seek, connect, accept);
3. **Data budget**—cumulative bytes transferred through mediated I/O paths;
4. **Operation budget**—total count of mediated resource operations; and
5. **Temporal expiry**—wall-clock deadline after which all mediated operations fail atomically.

Envelopes are implemented in A20OS, a research operating system with a dual-ABI architecture: a Linux compatibility layer (361 registered syscalls) runs unmodified musl-linked binaries alongside a native capability-aware ABI (136 syscalls) for trusted components. A kernel mediator (approximately 670 lines of C) intercepts every resource acquisition and consumption path, enforcing budget policies via per-handle shadow entries with direction-aware right checks.

1.2 Contributions

- A dual-ABI kernel architecture that attaches quantified budget policies to unmodified binaries at launch time (§3);
- An A1–A10 entry-point taxonomy covering 366 registered syscalls, mechanically verified against the syscall table via a drift gate (§4);
- A 15-scenario exercise suite (12 attack scenarios, 2 benign multi-process flows, 1 runtime audit) covering file, network, and IPC surfaces (§5);
- An exact-execution evaluation: 66 real malicious packages run unmodified on stock Node.js/CPython under A20OS—all 25 samples that reach the network at runtime are blocked at `socket()`, while reaching live C2 infrastructure undefended—plus an 87-sample canonical-replay study and 20 benign install footprints completing under budget (§5.3);
- A 20-cell pilot experiment across four defense arms quantifying the coarse-grained dilemma and showing envelopes resolve it (§5);
- A TLA+ model of the mediator state machine, exhaustively checked over a bounded state space (4,620 distinct states), plus core Lean 4 proofs covering monotone decay, transfer clamping, delegation bounding, and cross-envelope budget bounds (§6).

2 Background and Motivation

2.1 The Rewrite Mandate

Capsicum [1] introduced capability mode to FreeBSD, requiring programs to enter a restricted mode and operate exclusively through capability descriptors. Adopting Capsicum demands rewriting file access to use `openat(2)` with directory descriptors, replacing `execve(2)` with capability-aware variants, and restructuring privilege separation. These changes are practical only for programs designed with capability separation in mind.

Landlock [8] brought unprivileged access control to Linux through stacked filesystem rules. While Landlock does not require source modification, it operates on *paths*, not *objects*: a rule allowing writes to `/var/log` grants access to every file beneath that prefix regardless of ownership or type, and cannot express "at most 100 writes" or "expires in 300 seconds."

seccomp-BPF [9] filters system calls by number and argument predicates. It cannot distinguish between opening `/tmp/build/output.o` and `~/.ssh/id_rsa`—both are `openat(2)` calls with different path arguments, which BPF programs cannot reliably inspect due to pointer indirection.

The result is a dilemma we call the *coarse-grained trap*: path-based and syscall-number-based policies face a binary choice between permissiveness (which admits attacks) and restrictiveness (which blocks legitimate workflows). Our pilot experiments (§5) quantify this dilemma empirically.

2.2 Threat Model

We target *install-time attacks* in package managers (npm postinstall, pip setup.py, cargo build scripts), CI/CD runners, and plugin loading systems. In these scenarios, untrusted code executes with the user's full privileges during package installation. Empirical studies of malicious packages in npm and PyPI find that the majority activate during the install or import phase rather than during normal runtime execution [20,21,23].

The attacker controls the package content but not the OS configuration. The defender (user or CI operator) seeks to constrain the installation process without breaking legitimate packages. The defender can modify the *launch command* (adding envelope parameters) but cannot modify the *package source code*.

We do not address:

- Kernel vulnerabilities exploited to escape the mediation layer;
- Side-channel information leakage through shared microarchitectural state;
- Supply-chain compromise upstream of the package registry;
- Social engineering convincing users to weaken envelope policies.

2.3 Why Not Existing Linux Mechanisms?

A natural question is whether the same policy expressiveness could be deployed on commodity Linux without a new kernel. We survey the four closest mechanisms and the structural gap each leaves open.

rlimits and cgroups. POSIX resource limits [15] and cgroups v2 [16] are the budget mechanisms Linux already deploys. Both constrain *resource consumption* (file size, memory, CPU, I/O bandwidth) rather than *authority*: an rlimit cannot express "read but not write," neither mechanism reasons

about object types or descriptor transfers between processes, and neither offers temporal expiry with atomic revocation. Resource containers [14] established the value of kernel-visible resource principals; envelopes borrow that insight but budget *operations on authorities* rather than scheduler-visible consumption.

eBPF-LSM. BPF LSM (KRSI) [13] attaches custom policy to object-level security hooks and is the closest programmable neighbor. Three structural differences remain. First, *deployment privilege*: loading BPF LSM programs requires `CAP_BPF` and installs global policy, whereas envelopes attach unprivileged and per-launch, in the spirit of Landlock's self-restriction model. Second, *structural non-refreshability*: a BPF-map budget can be re-incremented by any component that can reach the map—the verifier guarantees memory safety, not that a policy never tops a counter back up. Envelope counters live in kernel-owned state, and the only mutation exposed to user space is decrement-by-charge (§3). Third, *io_uring*: fixed-file registrations bypass the file-based LSM hooks, and submission-point hooks cannot charge operations that execute later in kernel context; our mediation sits at the SQE execution point and fails closed on fixed files (§4). A privileged BPF-LSM reimplementation could approximate much of the envelope policy surface; we view such a Linux port as worthwhile future work and position envelopes as the unprivileged, deployment-time formulation with mechanically verified coverage.

Sandbox runtimes. gVisor [18] and WebAssembly [19] confine unmodified (resp. recompiled) code behind an interposed runtime. gVisor demonstrates that user-space ABI interception is deployable at the cost of a second system-call surface and non-trivial compatibility and performance overheads; neither substrate provides budget, expiry, or transfer-clamping dimensions. Envelopes mediate in-kernel at object granularity, so one policy engine covers both the Linux-ABI compatibility path and native capability-aware services.

3 Design

3.1 Dual-ABI Architecture

A20OS exposes two system-call interfaces sharing a common kernel core:

- **Linux ABI** (361 entries): binary-compatible with musl, enabling unmodified static binaries to execute normally.
- **Native ABI** (136 entries): capability-aware interface exposing handles, channels, and events for trusted services.

Both ABIs share the same VFS, memory manager, and scheduler. The key architectural property is that *every* resource acquisition passes through a common mediation layer regardless of which ABI issued the request. A capability envelope attaches to this mediation layer, applying budget policies transparently to whichever ABI the enveloped process uses.

Table 1: Escape-surface entry points.

ID	Path	Mediation
A1	open/openat	Type ∩ rights ∩ cap
A2	socket	Network rights + cap
A3	pipe/memfd/eventfd/timerfd/signalfd	Creation-time
A4	mmap (file-backed)	Map right + time
A5	shmat	MEMORY class check
A6	SCM_RIGHTS recv/send	Transfer clamp / propagation
A7	pidfd_getfd	Fresh acquisition
A8	/proc/self/fd reopen	Rights intersection
A9	io_uring fixed-file	Fail-closed
A10	io_uring SQE exec	Execution-point charge

3.2 Budget Lattice

Each envelope carries a policy structure defining per-class caps:

```
struct env_policy {
    uint32_t allowed_types;       /* class bitmask */
    uint64_t rights_cap[N];      /* per-type rights mask *
    uint64_t time_budget_ns;     /* temporal deadline */
    uint64_t op_budget;          /* total ops */
    uint64_t data_budget;        /* total bytes */
    uint32_t propagation_types;  /* outbound delegation */
};
```

All budget counters decrease monotonically through mediated operations. The mediator exposes no increment path: once set at creation time, budgets can only be consumed, never refreshed. This property—which we call *non-refreshability*—is enforced structurally: the only mutation on budget counters is decrement-by-charge, and no syscall increases any counter.

3.3 Entry-Point Taxonomy

We enumerate every path through which a process can acquire a new usable resource authority (Table 1). Each entry is assigned a unique identifier (A1–A10) and mapped to a specific kernel hook point where budget mediation fires.

Coverage is mechanically verified: a generator script parses the syscall table (365 entries), classifies each against the contract, and a drift gate fails whenever a new syscall is registered without explicit classification. Currently 20 entries are actively mediated, 46 are classified PLANNED-W2 (scheduled refinement), and 299 involve no resource authority.

3.4 Direction Enforcement

Each shadow handle tracks its granted rights as independent boolean fields (read, write, stat, seek). When an enveloped task performs an I/O operation, the mediator checks the operation's direction against the shadow's rights:

- READ-direction operations require the read bit;
- WRITE-direction operations require the write bit;
- Control operations (bind, listen, ioctl) skip direction checks.

This prevents a classic upgrade attack: opening a file read-only then reopening it through `/proc/self/fd/N` with write flags. The reopen inherits only the intersection of requested and original rights, so the write bit cannot be fabricated if absent from the source handle.

3.5 Transfer Clamping

When an enveloped process receives a descriptor via `SCM_RIGHTS` or `pidfd_getfd`, the received authority's rights are clamped to the intersection of the request and the receiver's policy cap:

$$\text{granted} = \text{request} \cap \text{cap}$$

An empty grant denies the transfer entirely. This ensures transferred authority never exceeds the receiver's policy ceiling, preventing both upgrade (beyond policy) and fabrication (beyond request).

4 Implementation

The mediator is implemented as a ∼670-line C module (`kernel/ipc/envelope.c`) integrated into A20OS's system-call dispatch layer. It maintains per-envelope state including the policy snapshot, live budget counters, and a shadow table keyed by global fd.

Control-plane syscalls (902–905) allow a supervisor to create envelopes, attach processes, trigger revocation, and query audit counters. The deployment pattern follows a create-fork-enter-execve model: the supervisor creates an envelope from a policy struct, forks, and the child enters before calling `execve` on the untrusted binary. Once entered, the attachment is monotone: there is no leave operation, and `execve` preserves the attachment.

USE-side hooks fire at six I/O families (read/write/readv/writev/ pread64/pwrite64) with per-family direction bits and conservative data-budget pre-charging. Vectorized variants pre-compute total segment lengths before mediation so denial fires before any partial transfer.

Socket operations mediate at three levels: creation (acquisition), control (bind/connect/listen ops charge), and data-plane (send/recv through separate hooks). Accepted connections inherit the listener's class but receive their own shadow entry clamped to the receiver's policy.

io_uring operations are mediated at the SQE execution point rather than the submission point, ensuring that asynchronous operations cannot bypass budget accounting after submission. Fixed-file registration is denied outright for enveloped tasks until a complete transfer-tracking design is validated.

A coverage matrix generator mechanically parses the syscall table and classifies each of 365 registered entries against the mediation contract. A drift gate fails compilation whenever a new syscall is registered without explicit classification, preventing silent gaps in the choke-point guarantee.

5 Evaluation

We evaluate capability envelopes along three axes: security efficacy (can attacks be blocked?), benign compatibility (do legitimate workloads still function?), and overhead (what is the mediation cost?).

All experiments run under QEMU system emulation (riscv64, 1 GiB RAM) on a host with AMD Ryzen 7735HS. We deliberately report this fixed, publicly reproducible configuration rather than hardware-specific numbers: neither the security conclusions nor the relative-overhead argument depends on the host, and emulator runs are exactly reproducible by others.

5.1 Attack Suite

We exercise 15 scenarios—12 attack scenarios each verifying a specific enforcement property, two benign multi-process flows, and one runtime invariant audit (Table 2). Every scenario runs in a forked worker inside the envelope; the parent verifies the expected outcome.

Every scenario passes: the mediator blocks all tested attacks while permitting benign operations within policy bounds. We emphasize that these are mechanism tests we authored: they establish that each enforcement property fires as designed, not adversarial completeness against an independent attacker; §5.3 evaluates against real malicious packages.

5.2 Pilot Experiment: Coarse-Grained Dilemma

We construct a five-scenario experiment comparing four defense arms (Table 3). Each cell forks a worker whose exit code encodes the observed outcome.

The results quantify the coarse-grained dilemma: LL-permissive admits all attacks (S0/A1/A2/A3/A4 all succeed); LL-strict blocks attacks but also kills the benign install (S0

Table 2: Attack suite results (all pass).

Scenario	Tests	Result
Type denial	socket() outside allowed_types	EPERM
Rights deny	O_WRONLY vs read-only cap	EACCES
Op budget	Third op after budget=2 exhausted	EACCES
Data budget	Write exceeding data_budget	EACCES
Expiry	Post-deadline read after lazy sweep	EACCES
Revoke+kill	Active revoke + KILL_ON_EXPIRE	SIGKILL
Reopen upgrade	/proc/self/fd/N write-after-RD-only	EACCES
SCM receive	Descriptor from non-allowed class	dropped
SCM send	propagation_types=0 sendmsg	EPERM
pidfd getfd	Cross-task fd theft	EPERM
shmat class	MEMORY outside allowed_types	EPERM
socket data plane	Over-budget sendto on AF_UNIX pair	EACCES
mksh flow	Full script under envelope	completed
pipe flow	3-process pipeline, one envelope	completed
Audit walk	Post-exercise invariant check	zero viol.

Table 3: Pilot experiment: five scenarios × four arms.

Scenario	NONE	LL-perm	LL-strict	ENV
S0 install	ok	ok	**blocked**	ok
A1 exfil	ok	ok	blocked	**blocked**
A2 loop	ran	ran	ran	**stopped**
A3 escape	ok	ok	blocked	pass*
A4 stall	kept	kept	kept	**denied**

*A3: path dimension outside envelope v1 scope; composes with Landlock for full coverage.

blocked). Only the envelope achieves both goals simultaneously: S0 completes while A1/A2/A4 are blocked. A3 (path escape) passes through the envelope v1 honestly (no path dimension), composing with Landlock for combined coverage. Each cell is a single deterministic run whose exit code encodes a permit/deny outcome that is deterministic by construction; we deliberately scope the claim to these five scenarios and do not extrapolate to package-installation workloads in general.

5.3 Corpus Evaluation: Real Malicious Packages

To move beyond self-authored scenarios, we evaluate against *real* malicious packages in two complementary ways: exact execution under stock interpreters, and canonical replay for broader coverage.

Table 4: Exact execution of 66 real malicious packages, NONE vs. ENVELOPE. "Reached net" counts samples whose payload issued a `socket()` (mediator counter).

	NONE	ENVELOPE
Reached network, attack proceeds	25	0
Reached network, blocked at `socket()`	0	25
No network attempt at runtime	41	41
Interpreter aborted by policy	—	0

Exact execution. From the DataDog malicious-software-packages dataset [24] we drew a seeded random sample of 100 packages (60 npm, 40 PyPI, from pools of 12,601 and 1,816). 67 samples carry an install-time entry point (34 npm lifecycle scripts, 33 PyPI `setup.py`); 66 executed to completion in our matrix. Samples run *unmodified* on stock Alpine Node.js 24.18.1 and CPython 3.12.14 inside A20OS (the `pynode` instance), each twice: bare (NONE) and wrapped by an install-time envelope (ENV: FILE+PIPE+event-queue+timer classes, no SOCKET, 100k-op / 256-MiB budgets) via a `create-fork-enter-execve` supervisor that also reads back the mediator's audit counters.

Of the 66 samples, 25 reached a network operation at runtime—the mediator's `acquire_deny_type` counter proves a `socket()` call occurred—and *all 25 were blocked* under the envelope (Table 4). Under NONE, the same payloads progressed to live network behavior: an exfiltration POST to a `oastify.com` listener that sent data before the connection broke, a completed TLS handshake to `webhook.site`, and DNS resolution of an AWS Lambda C2 hostname; under ENV, each died at `socket()` with EPERM. The remaining 41 samples never reached the network at runtime (payloads dormant, or crashing on interpreter version/environment differences in *both* arms)—a normal hit rate for detonation of aged samples [21]. No interpreter aborted under the envelope, and no benign behavior was denied.

Canonical replay for coverage. Separately, a static classifier (`tools/corpus/extract_corpus.py`) maps 87 of the 100 samples to a canonical behavior class (6 credential exfiltration, 25 network beaconing, 56 second-stage download) without executing them. Replaying each class's syscall sequence under the same two arms confirms the mechanism at scale: every attack goal succeeds under NONE (87/87, validating replay fidelity) and is blocked under the envelope (87/87), stable across three repetitions. The install footprints of 20 popular benign npm packages (120 files, 258 KiB of real tarball writes) complete under manifest-sized budgets with 2× headroom (20/20).

We stress the boundary of each method: replay covers more samples but executes canonical sequences, not the original code; exact execution runs the original code but only as far as aged, partially dormant payloads go in a fresh sandbox. The

Table 5: Mediation overhead (QEMU/TCG, directional).

Operation	off (ns)	on (ns)	Δ
open+close	79,587	82,834	+4.1%
read 64B	12,238	15,815	+29.2%
write 64B	32,049	37,731	+17.7%
lseek (ctl)	11,383	11,507	+1.1%
sendto+drain 64B	96,051	104,795	+9.1%

two agree on the claim that matters: whenever install-time code reaches for the network, the envelope denies it.

5.4 Real-Binary Compatibility

The G1 mksh-flow cell proves that a real static musl binary (/bin/mksh, ∼400KB) runs correctly inside an envelope across an execve boundary. The shell's internal openat/write operations are mediated and tracked, and the marker file is written correctly.

The G2 wget-blocked cell demonstrates TYPE-level enforcement blocking a real binary's network access. Under an envelope whose `allowed_types` excludes SOCKET, `wget` exits nonzero because its internal `socket()` call returns EPERM.

The G4 pipe-flow cell extends this to multi-process composition: an enveloped shell builds a three-stage pipeline using builtins only. Every segment inherits the shared envelope across fork, both `pipe2()` acquisitions are mediated (A3), and the final redirect's openat/write is charged – three processes, one envelope, zero unmediated steps.

5.5 Overhead

We measure per-operation latency with and without envelope activation using CLOCK_MONOTONIC sampled once per 20,000-iteration loop (Table 5).

The lseek control group shows noise-level variation (+1.1%), validating measurement methodology. Acquisition-path overhead is +4.1%; file use-path overhead ranges from +17.7% (write) to +29.2% (read). The socket data plane, measured as a paired `sendto` plus recvfrom drain over an AF_UNIX stream pair, adds +9.1%: transport cost dominates the mediated pair, so relative mediation overhead is lower than on the file path. Absolute deltas are ∼3.6μs (read) and ∼5.7μs (write) under TCG emulation; all comparisons are same-configuration.

6 Formal Verification

We verify the mediator's safety properties using two complementary approaches.

6.1 TLA+ Model Checking

We encode the mediator state machine in TLA+ (`Envelope.tla`, 199 lines) and exhaustively check it with TLC over a bounded state space (OpsMax=3, DataMax=4, ExpireAt=3, MaxTick=6, 3 tasks, 2 gfds, 2 types). Five invariants hold:

- TYPEALLOWED: shadow types within allowed_types;
- RIGHTSSUBCAP: shadow rights within class caps;
- OPSNONNEG / DATANONNEG: budget counters bounded;
- CLOCKBOUNDED: clock within model horizon.

Two temporal properties hold: NoResurrect (expired stays expired) and KillOnce (kill flag is one-way). TLC explores 27,829 states generating 4,620 distinct states at depth 14 in under one second.

6.2 Lean 4 Proofs

We formalize core budget lattice properties in Lean 4 (`BudgetLattice.lean`, 200 lines) using only core constructs (no Mathlib dependency). Thirteen theorems are proved with zero `sorry`, in four families:

- *Budget decay and expiry*: `deduct_le` (monotone decay), `deduct_strict` (strict positive decay), `decay_transitive` (chained spending), `latch_true` (one-way expiry latch);
- *Transfer clamping*: `and_field_le_cap` (clamped grant never exceeds the policy cap), `and_field_preserves` (legitimate authority survives clamping), `and_field_idem` (idempotent under repeated mediation); `xferred_wr/rd_needs_both` enforce direction on transferred shadows – a write- or read-pass implies both requester and policy granted the bit;
- *Delegation bounding*: `del_wr/rd_needs_both` – exercising a delegated right requires delegator grant AND receiver cap, so authority cannot escalate through the trust chain;
- *Cross-envelope budgets*: `scm_recv_le_receiver`, `scm_recv_le_sender` – receive-time charging must stay within min(sender, recv); our v1 prototype satisfies both bounds by construction (receipt draws one op from the receiver's own remaining budget and couples to no sender-side amount), so an incoming gift overdraws neither party.

6.3 From Models to Code

We do not claim refinement between the models and the C implementation. Instead, each model-checked property is tied to a single auditable code point in `kernel/ipc/envelope.c`, and—unusually—the same invariants are *re-checked at runtime*: the `env_audit` syscall walks every live envelope and re-verifies TYPEALLOWED, RIGHTSSUBCAP, and the budget bounds on live kernel state (the *Audit walk* row of Table 2). The correspondence is:

- TYPEALLOWED: the `allowed_types` gate in `env_mediate_acquire` and `env_mediate_class` precedes shadow installation;
- RIGHTSSUBCAP: `env_acquire_charge_install_locked` stores `rights & cap`, and the transfer path (`env_mediate_acquire_gfd`) stores `want & rights_by_class[kind]`;
- OPSNONNEG/DATANONNEG: every charge site checks the counter for zero (resp. sufficiency) *before* decrementing, so counters cannot underflow;
- NORESURRECT: the `expired` flag is set only by `env_mark_expired_locked` and has no clear path;
- KILLONCE: `kill_sent` is an atomic test-and-set shared by the expiry path and `env_revoke`;
- Monotone attachment: `env_enter` is a compare-and-swap from `NULL` only, so an enveloped task can neither switch envelopes nor leave.

The runtime audit turns the model-checked invariants into executable checks on real kernel state, catching divergence between the models and the implementation as a testable violation rather than a silent gap.

7 Related Work

Capability systems. KeyKOS [2] and EROS [3] established pure capability operating systems with persistent, delegatable authority, but required applications written against their object model. Capsicum [1] brought capability mode to a commodity UNIX at the cost of the rewrite mandate we target. Zircon [4] exposes handle-based objects in a production microkernel—our native ABI follows a similar shape—but its security story targets new userspace, not legacy Linux binaries. CHERI [5] provides hardware capabilities with architectural support; envelopes are software-only and operate on unmodified binaries.

Unprivileged self-restriction. OpenBSD's pledge [6] narrows the syscall surface by promise group but requires source modification and offers no budget dimensions; unveil [7] restricts path visibility but, like Landlock, cannot express volume or time. Landlock [8] provides unprivileged filesystem access control on Linux; our pilot experiment quantifies its coarse-grained dilemma. seccomp-BPF [9] filters by syscall number and scalar arguments but cannot inspect pointer-indirect arguments and carries no per-object state.

Mandatory access control and the LSM framework. The LSM framework [10] made Linux security policy pluggable; SELinux [11] and AppArmor [12] provide mandatory access control behind it, requiring administrator-installed global policy and, in the path-based AppArmor case, inheriting the path granularity we avoid. BPF LSM [13] adds programmability at the same hooks; §2.3 details the structural differences from envelopes (deployment privilege, non-refreshability, io_uring).

Resource accounting. Resource containers [14] introduced kernel-visible resource principals for consumption accounting; POSIX rlimits [15] and cgroups v2 [16] are their deployed descendants. All three meter consumption rather than authority and provide neither direction enforcement nor transfer clamping (§2.3).

Sandbox runtimes. Native Client [17] and WebAssembly [19] confine code through software fault isolation at the cost of recompilation or restricted toolchains. gVisor [18] interposes a user-space kernel for unmodified containers, trading a second system-call surface and compatibility overhead for isolation; it has no budget or expiry dimensions.

Supply-chain security. Measurement studies document the scale and install-time character of open-source supply-chain attacks [20–23]. On the defense side, Latch [25] infers install-time manifests via strace and enforces them via AppArmor, and Leash [26] sandboxes installer scripts using FreeBSD Capsicum with a user-space supervisor. Both operate at coarser granularity than object-level mediation and lack budget dimensions.

Formal verification of OS kernels. seL4 [27] achieved full functional-correctness verification of a microkernel; CertiKOS [28] verified concurrent kernels via layered refinement; Serval [29] automated verification of systems code through symbolic evaluation. We target specific security properties of one subsystem rather than whole-kernel refinement, using TLA+ model checking [30] and Lean 4 proofs [31] as complementary tools, with runtime re-verification bridging models and code (§6.3).

8 Discussion and Limitations

Scope. Envelopes constrain *authority lifetime and volume*, not information-flow control. Data willingly written to permitted destinations persists beyond envelope expiry. This

matches the threat model: the mechanism bounds unauthorized acquisition, operation volume, and time window—not post-hoc recall.

Path dimension. Envelope v1 mediates by resource class, not by filesystem path. Path-scoped policies compose orthogonally with envelopes (e.g., Landlock for WHERE, envelope for HOW-MUCH/HOW-LONG).

Performance. Overhead figures come from the fixed QEMU/TCG configuration above; TCG amplifies syscall-dense loops, so absolute values should be read as same-configuration comparisons rather than native-hardware costs. Application-level end-to-end benchmarks remain future work.

Evaluation breadth. The corpus evaluation (§5.3) grounds the security claim in real malicious packages executed on stock interpreters, but two boundaries remain. First, aged samples detonate imperfectly: 41 of 66 payloads never reached the network in *either* arm (dormancy, interpreter drift, dead infrastructure), so our block rate is measured on the 25 samples that demonstrably attacked. Second, all 25 were network-dependent; path-only classes (persistence writes, credential reads without exfiltration) are outside envelope v1's scope and compose with Landlock, as in the pilot's A3. A privileged BPF-LSM port of the envelope policy surface would enable same-host comparison against commodity Linux (§2.3).

PLANNED-W2 surfaces. Forty-six syscalls are classified PLANNED-W2 (unmediated but enumerated). Key items include socket data-plane charging (sendto/recvfrom), mmap file-backed mapping, and SysV IPC creation. Each is tracked in the coverage matrix with assigned W2 line items.

9 Conclusion

We presented capability envelopes: a kernel-level mechanism that attaches quantified budget policies to unmodified Linux binaries at deployment time. Unlike prior capability systems that require source modification, envelopes exploit the asymmetry that budgets are deployable even when permissions are not. A dual-ABI kernel architecture enables transparent enforcement across 366 registered syscalls with direction-aware right checks, conservative data-budget pre-charging, and mechanical coverage verification.

Our pilot evaluation demonstrates that envelopes resolve the coarse-grained dilemma that plagues path-based policies *within the tested scenarios*: the envelope arm is the only configuration that both completes the benign installation and blocks the tested attack classes, through type denial, rights clamping, and budget exhaustion. Exact execution of 66 real malicious packages on stock interpreters confirms the result at corpus scale: every sample that reached the network (25/25) was denied at `socket()`, while undefended payloads reached live C2 infrastructure; an 87-sample canonical replay shows the same boundary. Overhead under emulation is +4.1% on the acquisition path and +17.7–29.2% on the file use path. A TLA+ model exhaustively verifies safety over a bounded state space, Lean proofs establish core lattice properties without external dependencies, and a runtime audit re-checks the model-checked invariants on live kernel state.

Availability. The A20OS kernel, the envelope mediator, the attack-suite and pilot harnesses, the TLA+ model, and the Lean 4 proofs are open source and publicly reproducible under the fixed QEMU configuration reported in §5.

References

[1] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, "Capsicum: Practical capabilities for UNIX," in *Proc. 19th USENIX Security Symposium*, 2010.

[2] N. Hardy, "The KeyKOS architecture," *ACM Operating Systems Review*, 19(4):8–25, 1985.

[3] J. S. Shapiro, J. M. Smith, and D. J. Farber, "EROS: A fast capability system," in *Proc. 17th ACM Symposium on Operating Systems Principles (SOSP)*, 1999.

[4] Google, "Fuchsia: Zircon kernel and handle-based object model," Fuchsia project documentation, 2023. https://fuchsia.dev/fuchsia-src/concepts/kernel.

[5] J. Woodruff, R. N. M. Watson, D. Chisnall, S. W. Moore, J. Anderson, B. Davis, B. Laurie, P. G. Neumann, R. Norton, and M. Roe, "The CHERI capability model: Revisiting RISC in an age of risk," in *Proc. 41st International Symposium on Computer Architecture (ISCA)*, 2014.

[6] OpenBSD, "`pledge(2)` — restrict system operations," OpenBSD Programmer's Manual, 2015.

[7] OpenBSD, "`unveil(2)` — unveil parts of a directory tree," OpenBSD Programmer's Manual, 2018.

[8] M. Salaün, "Landlock: Unprivileged access control," Linux kernel documentation, v6.2+, 2023.

[9] W. Drewry, "Seccomp BPF (SECure COMPuting with filters)," Linux kernel documentation, v3.5+, 2012.

[10] C. Wright, C. Cowan, S. Smalley, J. Morris, and G. Kroah-Hartman, "Linux Security Modules: General security support for the Linux kernel," in *Proc. 11th USENIX Security Symposium*, 2002.

[11] P. Loscocco and S. Smalley, "Integrating flexible support for security policies into the Linux operating system," in *Proc. USENIX Annual Technical Conference (Freenix Track)*, 2001.

[12] C. Cowan, S. Beattie, G. Kroah-Hartman, C. Pu, P. Wagle, and V. Gligor, "SubDomain: Parsimonious server security," in *Proc. 14th USENIX Conference on System Administration (LISA)*, 2000.

[13] K. P. Singh, "Kernel Runtime Security Instrumentation (BPF LSM)," Linux kernel documentation, 2019.

[14] G. Banga, P. Druschel, and J. C. Mogul, "Resource containers: A new facility for resource management in server systems," in *Proc. 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 1999.

[15] IEEE and The Open Group, "`setrlimit()` — control maximum resource consumption," POSIX.1-2017, 2017.

[16] The Linux kernel documentation, "Control Group v2," online. https://docs.kernel.org/admin-guide/cgroup-v2.html.

[17] B. Yee, D. Sehr, G. Dardyk, J. B. Chen, R. Muth, T. Ormandy, S. Okasaka, N. Narula, and N. Fullagar, "Native Client: A sandbox for portable, untrusted x86 native code," in *Proc. 30th IEEE Symposium on Security and Privacy (S&P)*, 2009.

[18] Google, "gVisor: A container sandbox runtime," online, 2019. https://gvisor.dev/docs.

[19] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, "Bringing the Web up to speed with WebAssembly," in *Proc. 38th ACM Conference on Programming Language Design and Implementation (PLDI)*, 2017.

[20] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Proc. 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2020.

[21] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, "Towards measuring supply chain attacks on package managers for interpreted languages," in *Proc. 28th Network and Distributed System Security Symposium (NDSS)*, 2021.

[22] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *Proc. 28th USENIX Security Symposium*, 2019.

[23] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of attacks on open-source software supply chains," in *Proc. 44th IEEE Symposium on Security and Privacy (S&P)*, 2023.

[24] DataDog, "Malicious software packages dataset," online, 2023–2026. https://github.com/DataDog/malicious-software-packages-dataset.

[25] Z. Luo et al., "Latch: Preventing install-time attacks in npm with lightweight manifest checking," in *Proc. 17th ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2022.

[26] S. Jadidi and J. Anderson, "Leash: A transparent capability-based sandboxing supervisor for Unix," in *Proc. 11th International Conference on Information Systems Security and Privacy (ICISSP)*, 2025.

[27] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood, "seL4: Formal verification of an OS kernel," in *Proc. 22nd ACM Symposium on Operating Systems Principles (SOSP)*, 2009.

[28] R. Gu, Z. Shao, H. Chen, X. N. Wu, J. Kim, V. Sjöberg, and D. Costanzo, "CertiKOS: An extensible architecture for building certified concurrent OS kernels," in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.

[29] L. Nelson, J. Bornholt, R. Gu, A. Baumann, E. Torlak, and X. Wang, "Scaling symbolic evaluation for automated verification of systems code with Serval," in *Proc. 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[30] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*, Addison-Wesley, 2002.

[31] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. 28th International Conference on Automated Deduction (CADE)*, 2021.